

Mảng hậu tố

Xu Zhilei

Trường Trung học số 1 thành phố Vu Hồ, An Huy

2004

Nguồn gốc: Suffix array

IOI China candidate team papers / 2004 / Xu Zhilei.pdf

Abstract

Bài viết giới thiệu các khái niệm cơ bản, phương pháp và ứng dụng của mảng hậu tố.

Trước hết bài viết trình bày thuật toán nhân đôi xây dựng mảng hậu tố với độ phức tạp $\mathcal{O}(n \log n)$. Tiếp theo là cách tính tiền tố chung dài nhất LCP (*Longest Common Prefix*) đi kèm với mảng hậu tố, cùng một thuật toán tuyến tính để tính mảng height, tức mảng ghi các giá trị LCP của hai hậu tố kề nhau trong thứ tự mảng hậu tố. Để người đọc có cảm nhận cụ thể về cách dùng mảng hậu tố, bài viết cũng nêu hai ví dụ ứng dụng: so khớp nhiều mẫu trên một xâu cố định, với thời gian $\mathcal{O}(m + \log n)$ cho mỗi lần so khớp, và tìm xâu con đối xứng dài nhất, với thời gian $\mathcal{O}(n \log n)$. Cuối cùng bài viết so sánh mảng hậu tố với cây hậu tố.

Từ khóa: xâu; hậu tố; quan hệ so sánh tiền tố k ; mảng hậu tố; mảng hạng; cây hậu tố; thuật toán nhân đôi; sắp xếp cơ sở; tiền tố chung dài nhất; bài toán RMQ; so khớp mẫu; xâu đối xứng; xâu con đối xứng dài nhất.

1 Mở đầu

Trong xử lý xâu, cây hậu tố và mảng hậu tố đều là những công cụ rất mạnh. Cây hậu tố được biết đến nhiều hơn, còn tài liệu tiếng Trung về mảng hậu tố khá hiếm. Thực ra mảng hậu tố là một thay thế rất tinh tế cho cây hậu tố: nó dễ lập trình hơn cây hậu tố, có thể thực hiện nhiều chức năng của cây hậu tố với độ phức tạp thời gian không kém quá nhiều, và dùng ít bộ nhớ hơn nhiều. Có thể nói trong các kỳ thi tin học, mảng hậu tố thực dụng hơn cây hậu tố. Vì vậy bài viết này giới thiệu các khái niệm cơ bản, cách xây dựng mảng hậu tố, cách xây dựng mảng tiền tố chung dài nhất đi kèm với nó, rồi bàn về ứng dụng của mảng hậu tố qua một số ví dụ.

2 Khái niệm cơ bản

Trước hết ta làm rõ một vài định nghĩa cần thiết.

Bảng chữ cái. Một bảng chữ cái Σ là một tập hợp đã được đặt quan hệ thứ tự toàn phần. Nói cách khác, với hai phần tử khác nhau α và β bất kỳ trong Σ , luôn có thể so sánh chúng: hoặc $\alpha < \beta$, hoặc $\beta < \alpha$, tức $\alpha > \beta$. Các phần tử trong Σ được gọi là ký tự.

Xâu. Một xâu S là một mảng thu được bằng cách xếp tuần tự n ký tự. Số n gọi là độ dài của S , ký hiệu $\text{len}(S)$. Ký tự thứ i của S được ký hiệu là $S[i]$.

Xâu con. Xâu con $S[i..j]$ của xâu S , với $i \leq j$, là đoạn từ vị trí i đến vị trí j trong S , tức xâu tạo bởi dãy ký tự

$$S[i], S[i+1], \dots, S[j].$$

Hậu tố. Hậu tố là một xâu con đặc biệt bắt đầu từ một vị trí i nào đó và kéo dài tới cuối xâu. Hậu tố của S bắt đầu ở i được ký hiệu

$$\text{Suffix}(S, i) = S[i.. \text{len}(S)].$$

So sánh kích thước của hai xâu nghĩa là so sánh theo thứ tự từ điển thông thường. Với hai xâu u, v , cho i bắt đầu từ 1 và lần lượt so sánh $u[i]$ với $v[i]$. Nếu $u[i] = v[i]$ thì tăng i lên 1; nếu $u[i] < v[i]$ thì $u < v$; nếu $u[i] > v[i]$ thì $u > v$, tức $v < u$, và quá trình so sánh kết thúc. Nếu đến khi $i > \text{len}(u)$ hoặc $i > \text{len}(v)$ mà vẫn chưa phân định được, thì nếu $\text{len}(u) < \text{len}(v)$ ta coi $u < v$, nếu $\text{len}(u) = \text{len}(v)$ ta coi $u = v$, còn nếu $\text{len}(u) > \text{len}(v)$ ta coi $u > v$.

Từ định nghĩa này, hai hậu tố bắt đầu ở hai vị trí khác nhau của S không thể bằng nhau, vì điều kiện cần để $u = v$ là $\text{len}(u) = \text{len}(v)$, điều không thể xảy ra ở đây.

Từ nay ta cố định một bảng chữ cái Σ và một xâu S . Đặt $\text{len}(S) = n$, và giả sử $S[n] = \$$, tức S kết thúc bằng một ký tự đặc biệt $\$$, nhỏ hơn mọi ký tự trong Σ . Ngoài $S[n]$, mọi ký tự khác của S đều thuộc Σ . Với xâu S đã cố định, hậu tố bắt đầu tại vị trí i được viết gọn là $\text{Suffix}(i)$, bỏ tham số S .

Mảng hậu tố. Mảng hậu tố SA là một mảng một chiều lưu một hoán vị của $1..n$:

$$\text{SA}[1], \text{SA}[2], \dots, \text{SA}[n],$$

và thỏa

$$\text{Suffix}(\text{SA}[i]) < \text{Suffix}(\text{SA}[i+1]), \quad 1 \leq i < n.$$

Nói cách khác, ta sắp xếp n hậu tố của S theo thứ tự tăng dần, rồi ghi lần lượt vị trí bắt đầu của các hậu tố đã sắp xếp vào SA.

Mảng hạng. Mảng hạng là $\text{Rank} = \text{SA}^{-1}$. Nghĩa là nếu $\text{SA}[i] = j$ thì $\text{Rank}[j] = i$. Dễ thấy $\text{Rank}[i]$ lưu “hạng” của $\text{Suffix}(i)$ trong thứ tự tăng dần của toàn bộ các hậu tố.

3 Phương pháp xây dựng

Làm thế nào để xây dựng mảng hậu tố? Cách trực tiếp và đơn giản nhất dĩ nhiên là coi các hậu tố của S như những xâu thông thường, rồi sắp xếp chúng tăng dần bằng một thuật toán sắp xếp xâu tổng quát.

Dễ thấy cách làm ấy vụng về, vì nó không tận dụng mối liên hệ chặt chẽ giữa các hậu tố, nên hiệu suất khó có thể cao. Ngay cả khi dùng *Multi-key Quick Sort*, một thuật toán khá hiệu quả trong sắp xếp xâu, độ phức tạp thời gian trường hợp xấu nhất vẫn là $\mathcal{O}(n^2)$, không đáp ứng yêu cầu của ta.

Dưới đây là thuật toán nhân đôi (*Doubling Algorithm*). Nó tận dụng đầy đủ quan hệ giữa các hậu tố và hạ thành công độ phức tạp xây dựng mảng hậu tố trong trường hợp xấu nhất xuống $\mathcal{O}(n \log n)$.

Với một xâu u , định nghĩa tiền tố k của u là

$$u_k = \begin{cases} u[1..k], & \text{len}(u) \geq k, \\ u, & \text{len}(u) < k. \end{cases}$$

Định nghĩa các quan hệ so sánh tiền tố k : $<_k$, $=_k$ và $\leq_k$. Với hai xâu u và v ,

$$u <_k v \iff u_k < v_k,$$

$$u =_k v \iff u_k = v_k,$$

$$u \leq_k v \iff u_k \leq v_k.$$

Trực quan, các ký hiệu so sánh có chỉ số k này nghĩa là so sánh từ điển trên k ký tự đầu của hai xâu. Điểm đặc biệt là khi so sánh lớn hơn hoặc nhỏ hơn, nếu độ dài một xâu chưa tới k thì cũng không sao, miễn là ta đã phân định được xâu thứ nhất lớn hơn hoặc nhỏ hơn xâu thứ hai trước khi việc so sánh k ký tự kết thúc.

Từ tính chất của phép so sánh tiền tố, ta có các tính chất rất quan trọng sau.

Tính chất 1.1. Với $k \geq n$,

$$\text{Suffix}(i) <_k \text{Suffix}(j) \iff \text{Suffix}(i) < \text{Suffix}(j).$$

Tính chất 1.2.

$$\text{Suffix}(i) =_{2k} \text{Suffix}(j)$$

tương đương với

$$\text{Suffix}(i) =_k \text{Suffix}(j) \quad \text{và} \quad \text{Suffix}(i+k) =_k \text{Suffix}(j+k).$$

Tính chất 1.3.

$$\text{Suffix}(i) <_{2k} \text{Suffix}(j)$$

tương đương với

$$\text{Suffix}(i) <_k \text{Suffix}(j) \quad \text{hoặc} \quad (\text{Suffix}(i) =_k \text{Suffix}(j) \text{ và } \text{Suffix}(i+k) <_k \text{Suffix}(j+k)).$$

Ở đây có một vấn đề: khi $i+k > n$ hoặc $j+k > n$, biểu thức $\text{Suffix}(i+k)$ hoặc $\text{Suffix}(j+k)$ không được định nghĩa rõ ràng. Thực tế ta không cần xét riêng trường hợp này, vì khi đó độ dài của $\text{Suffix}(i)$ hoặc $\text{Suffix}(j)$ không vượt quá k , tức tiền tố k của nó kết thúc bằng $\$$. Do đó kết quả so sánh tiền tố k không thể là bằng nhau: k ký tự đầu đã đủ để phân định lớn nhỏ, nên các biểu thức phía sau tự nhiên bị bỏ qua. Đây cũng là tác dụng đặc biệt của việc quy định S kết thúc bằng $\$$.

Định nghĩa mảng hậu tố k , ký hiệu SA_k , là mảng lưu một hoán vị của $1..n$:

$$\text{SA}_k[1], \text{SA}_k[2], \dots, \text{SA}_k[n],$$

sao cho

$$\text{Suffix}(\text{SA}_k[i]) \leq_k \text{Suffix}(\text{SA}_k[i+1]), \quad 1 \leq i < n.$$

Nói cách khác, ta sắp xếp toàn bộ hậu tố theo quan hệ so sánh tiền tố k , rồi ghi lần lượt vị trí bắt đầu của các hậu tố đã sắp xếp vào SA_k .

Định nghĩa mảng hạng k , ký hiệu Rank_k . $\text{Rank}_k[i]$ là hạng của $\text{Suffix}(i)$ trong thứ tự tăng dần theo quan hệ tiền tố k , tức

$$1 + \#\{j \mid \text{Suffix}(j) <_k \text{Suffix}(i)\}.$$

Từ SA_k có thể dễ dàng tính Rank_k trong thời gian $\mathcal{O}(n)$.

Giả sử đã có SA_k và Rank_k . Khi đó ta có thể thuận tiện tính SA_{2k} và Rank_{2k} . Theo Tính chất 1.2 và 1.3, quan hệ so sánh tiền tố $2k$ có thể được biểu diễn tương đương bằng hằng số phép so sánh tiền tố k ; còn mảng Rank_k thực chất cho ta cách thực hiện phép so sánh $<_k$ và $=_k$ trong thời gian hằng số:

$$\text{Suffix}(i) <_k \text{Suffix}(j) \iff \text{Rank}_k[i] < \text{Rank}_k[j],$$

$$\text{Suffix}(i) =_k \text{Suffix}(j) \iff \text{Rank}_k[i] = \text{Rank}_k[j].$$

Vì vậy so sánh $\text{Suffix}(i)$ và $\text{Suffix}(j)$ theo quan hệ tiền tố $2k$ có thể làm trong thời gian hằng số. Khi đó sắp xếp mọi hậu tố theo quan hệ $\leq_{2k}$ không khác gì sắp xếp thông thường: mỗi $\text{Suffix}(i)$ có một khóa chính $\text{Rank}_k[i]$ và một khóa phụ $\text{Rank}_k[i + k]$. Nếu dùng quicksort hoặc một thuật toán sắp xếp $\mathcal{O}(n \log n)$ tương tự, thì xây dựng SA_{2k} từ SA_k và Rank_k mất $\mathcal{O}(n \log n)$. Cách thông minh hơn là dùng sắp xếp cơ sở, khi đó độ phức tạp là $\mathcal{O}(n)$. Sau khi tính SA_{2k} , ta tính được Rank_{2k} từ SA_{2k} trong thời gian $\mathcal{O}(n)$. Do đó từ $\text{SA}_k, \text{Rank}_k$ suy ra $\text{SA}_{2k}, \text{Rank}_{2k}$ có thể hoàn thành trong thời gian $\mathcal{O}(n)$.

Còn lại một câu hỏi: làm thế nào xây dựng SA_1 và Rank_1 ? Việc này rất đơn giản, vì các phép $<_1, =_1$ và $\leq_1$ thực chất chỉ so sánh ký tự đầu của xâu. Vì thế chỉ cần sắp xếp các hậu tố theo ký tự đầu của chúng là có SA_1 . Ta có thể dùng quicksort, với độ phức tạp $\mathcal{O}(n \log n)$.

Như vậy có thể tính SA_1 và Rank_1 trong thời gian $\mathcal{O}(n \log n)$. Từ đó, ta tính $\text{SA}_2, \text{Rank}_2$ trong $\mathcal{O}(n)$, rồi $\text{SA}_4, \text{Rank}_4$ trong $\mathcal{O}(n)$, và cứ tiếp tục:

$$\text{SA}_2, \text{Rank}_2; \quad \text{SA}_4, \text{Rank}_4; \quad \text{SA}_8, \text{Rank}_8; \quad \dots$$

cho đến $\text{SA}_m, \text{Rank}_m$, trong đó $m = 2^k$ và $m \geq n$. Theo Tính chất 1.1, SA_m tương đương với SA . Tổng cộng cần thực hiện $\log n$ lần quá trình $\mathcal{O}(n)$, do đó

có thể tính mảng hậu tố SA và mảng hạng Rank trong $\mathcal{O}(n \log n)$.

4 Tiền tố chung dài nhất

Bây giờ mảng hậu tố SA của một xâu S đã có thể tính trong thời gian $\mathcal{O}(n \log n)$. Dùng SA , ta đã làm được nhiều việc, chẳng hạn so khớp mẫu trong thời gian $\mathcal{O}(m \log n)$, trong đó m, n lần lượt là độ dài mẫu và xâu cần so khớp. Nhưng để phát huy đầy đủ sức mạnh của mảng hậu tố, ta còn cần một công cụ phụ: tiền tố chung dài nhất, hay LCP (*Longest Common Prefix*).

Với hai xâu u, v , định nghĩa

$$\text{lcp}(u, v) = \max\{i \mid u =_i v\}.$$

Nói cách khác, ta so sánh lần lượt các ký tự tương ứng từ đầu hai xâu, và lấy vị trí lớn nhất mà các ký tự tương ứng còn liên tục bằng nhau. Giá trị này được gọi là độ dài tiền tố chung dài nhất của hai xâu.

Với các số nguyên dương i, j , định nghĩa

$$\text{LCP}(i, j) = \text{lcp}(\text{Suffix}(\text{SA}[i]), \text{Suffix}(\text{SA}[j])),$$

trong đó i, j đều thuộc $1..n$. $\text{LCP}(i, j)$ chính là độ dài tiền tố chung dài nhất của hậu tố thứ i và hậu tố thứ j trong mảng hậu tố.

Về LCP, có hai tính chất hiển nhiên:

$$\text{LCP}(i, j) = \text{LCP}(j, i),$$

$$\text{LCP}(i, i) = \text{len}(\text{Suffix}(\text{SA}[i])) = n - \text{SA}[i] + 1.$$

Ý nghĩa của hai tính chất này là khi tính $\text{LCP}(i, j)$, ta chỉ cần xét trường hợp $i < j$: nếu $i > j$ thì đổi chỗ i, j , còn nếu $i = j$ thì trực tiếp trả về $n - \text{SA}[i] + 1$.

Tính trực tiếp $\text{LCP}(i, j)$ theo định nghĩa bằng cách so sánh tuần tự các ký tự tương ứng rõ ràng rất kém hiệu quả, với thời gian $\mathcal{O}(n)$. Vì vậy ta cần tiền xử lý thích hợp để giảm độ phức tạp của mỗi lần tính LCP.

Phân tích kỹ hơn, ta thấy hàm LCP có một tính chất rất đẹp:

$$\text{nếu } i < j, \quad \text{LCP}(i, j) = \min\{\text{LCP}(k - 1, k) \mid i + 1 \leq k \leq j\}.$$

Ta gọi đây là **Định lý LCP**. Để chứng minh định lý này, trước hết chứng minh bổ đề sau.

Bổ đề LCP. Với mọi $1 \leq i < j < k \leq n$,

$$\text{LCP}(i, k) = \min\{\text{LCP}(i, j), \text{LCP}(j, k)\}.$$

Chứng minh. Đặt $p = \min\{\text{LCP}(i, j), \text{LCP}(j, k)\}$. Khi đó $\text{LCP}(i, j) \geq p$ và $\text{LCP}(j, k) \geq p$. Đặt

$$u = \text{Suffix}(\text{SA}[i]), \quad v = \text{Suffix}(\text{SA}[j]), \quad w = \text{Suffix}(\text{SA}[k]).$$

Từ $u =_{\text{LCP}(i, j)} v$ suy ra $u =_p v$; tương tự $v =_p w$. Vì thế $\text{Suffix}(\text{SA}[i]) =_p \text{Suffix}(\text{SA}[k])$, tức

$$\text{LCP}(i, k) \geq p. \quad (1)$$

Giả sử ngược lại $\text{LCP}(i, k) = q > p$. Khi đó

$$u[1] = w[1], u[2] = w[2], \dots, u[q] = w[q].$$

Mặt khác, $\min\{\text{LCP}(i, j), \text{LCP}(j, k)\} = p$ cho biết

$$u[p+1] \neq v[p+1] \quad \text{hoặc} \quad v[p+1] \neq w[p+1].$$

Đặt

$$x = u[p+1], \quad y = v[p+1], \quad z = w[p+1].$$

Do $u < v < w$ theo thứ tự của mảng hậu tố, hiển nhiên $x \leq y \leq z$. Lại vì $p < q$ nên $p+1 \leq q$, suy ra $x = z$. Do đó $x = y = z$, mâu thuẫn với $u[p+1] \neq v[p+1]$ hoặc $v[p+1] \neq w[p+1]$. Vậy $q > p$ không thể xảy ra, tức

$$\text{LCP}(i, k) \leq p. \quad (2)$$

Từ (1) và (2), $\text{LCP}(i, k) = p = \min\{\text{LCP}(i, j), \text{LCP}(j, k)\}$. Bổ đề được chứng minh.

Từ đây chứng minh Định lý LCP như sau. Khi $j - i = 1$ và $j - i = 2$, kết luận hiển nhiên đúng. Giả sử Định lý LCP đúng khi $j - i = m$. Xét trường hợp $j - i = m + 1$. Theo Bổ đề LCP,

$$\text{LCP}(i, j) = \min\{\text{LCP}(i, i+1), \text{LCP}(i+1, j)\}.$$

Vì $j - (i+1) \leq m$, theo giả thiết quy nạp,

$$\text{LCP}(i+1, j) = \min\{\text{LCP}(k-1, k) \mid i+2 \leq k \leq j\}.$$

Do đó khi $j - i = m + 1$, ta vẫn có

$$\begin{aligned} \text{LCP}(i, j) &= \min\{\text{LCP}(i, i+1), \min\{\text{LCP}(k-1, k) \mid i+2 \leq k \leq j\}\} \\ &= \min\{\text{LCP}(k-1, k) \mid i+1 \leq k \leq j\}. \end{aligned}$$

Theo quy nạp toán học, Định lý LCP đúng.

Từ Định lý LCP suy ra ngay một hệ quả tất yếu:

$$\text{với } i \leq j < k, \quad \text{LCP}(j, k) \geq \text{LCP}(i, k).$$

Định nghĩa mảng một chiều height bởi

$$\text{height}[i] = \text{LCP}(i-1, i), \quad 1 < i \leq n, \quad \text{và} \quad \text{height}[1] = 0.$$

Theo Định lý LCP,

$$\text{LCP}(i, j) = \min\{\text{height}[k] \mid i+1 \leq k \leq j\}.$$

Nói cách khác, tính $LCP(i, j)$ tương đương với hỏi giá trị nhỏ nhất trong mảng height trên đoạn chỉ số từ $i + 1$ đến j . Nếu mảng height cố định, đây chính là bài toán kinh điển RMQ (*Range Minimum Query*).

Bài toán RMQ có thể được tiền xử lý bằng cây đoạn hoặc cây sắp xếp tính trong thời gian $\mathcal{O}(n \log n)$, rồi mỗi truy vấn mất $\mathcal{O}(\log n)$. Cách tốt hơn là thuật toán chuẩn cho RMQ, có thể tiền xử lý trong thời gian $\mathcal{O}(n)$, sau đó trả lời mỗi truy vấn trong thời gian hằng số.

Với một xâu cố định S , mảng height rõ ràng là cố định. Chỉ cần ta tính height hiệu quả, sau khi tiền xử lý bằng phương pháp RMQ, mỗi lần tính $LCP(i, j)$ chỉ mất thời gian hằng số. Như vậy chỉ còn một vấn đề: làm thế nào tính mảng height càng hiệu quả càng tốt?

Từ kinh nghiệm tính mảng hậu tố, ta không nên coi n hậu tố là những xâu thông thường không liên quan tới nhau, mà nên tận dụng tối đa quan hệ giữa chúng. Dưới đây là một tính chất rất hữu ích.

Để tiện mô tả, đặt

$$h[i] = \text{height}[\text{Rank}[i]], \quad \text{tức là} \quad \text{height}[i] = h[\text{SA}[i]].$$

Mảng h thỏa tính chất sau.

Tính chất 3. Với $i > 1$ và $\text{Rank}[i] > 1$, luôn có

$$h[i] \geq h[i - 1] - 1.$$

Để chứng minh Tính chất 3, ta cần nêu rõ hai sự thật sau. Giả sử

$$i < n, \quad j < n, \quad \text{lcp}(\text{Suffix}(i), \text{Suffix}(j)) > 1.$$

Khi đó:

$$\text{Fact 1. } \text{Suffix}(i) < \text{Suffix}(j) \iff \text{Suffix}(i + 1) < \text{Suffix}(j + 1),$$

$$\text{Fact 2. } \text{lcp}(\text{Suffix}(i + 1), \text{Suffix}(j + 1)) = \text{lcp}(\text{Suffix}(i), \text{Suffix}(j)) - 1.$$

Nhìn qua có vẻ kỳ diệu, nhưng thật ra rất tự nhiên. Điều kiện $\text{lcp}(\text{Suffix}(i), \text{Suffix}(j)) > 1$ cho biết ký tự đầu của $\text{Suffix}(i)$ và $\text{Suffix}(j)$ bằng nhau; gọi ký tự đó là α . Khi đó $\text{Suffix}(i)$ tương đương với α nối trước $\text{Suffix}(i + 1)$, còn $\text{Suffix}(j)$ tương đương với α nối trước $\text{Suffix}(j + 1)$. Khi so sánh $\text{Suffix}(i)$ với $\text{Suffix}(j)$, ký tự đầu α chắc chắn bằng nhau, nên phần còn lại tương đương với việc so sánh $\text{Suffix}(i + 1)$ và $\text{Suffix}(j + 1)$. Vì vậy Fact 1 đúng; Fact 2 được chứng minh tương tự.

Bây giờ chứng minh Tính chất 3. Khi $h[i - 1] \leq 1$, kết luận hiển nhiên vì

$$h[i] \geq 0 \geq h[i - 1] - 1.$$

Khi $h[i - 1] > 1$, tức $\text{height}[\text{Rank}[i - 1]] > 1$, ta thấy $\text{Rank}[i - 1] > 1$ vì $\text{height}[1] = 0$. Đặt

$$j = i - 1, \quad k = \text{SA}[\text{Rank}[j] - 1].$$

Rõ ràng $\text{Suffix}(k) < \text{Suffix}(j)$. Từ

$$h[i - 1] = \text{lcp}(\text{Suffix}(k), \text{Suffix}(j)) > 1 \quad \text{và} \quad \text{Suffix}(k) < \text{Suffix}(j),$$

theo Fact 2,

$$\text{lcp}(\text{Suffix}(k + 1), \text{Suffix}(i)) = h[i - 1] - 1.$$

Theo Fact 1, $\text{Rank}[k + 1] < \text{Rank}[i]$, tức $\text{Rank}[k + 1] \leq \text{Rank}[i] - 1$. Do đó, theo hệ quả LCP,

$$\begin{aligned} \text{LCP}(\text{Rank}[i] - 1, \text{Rank}[i]) &\geq \text{LCP}(\text{Rank}[k + 1], \text{Rank}[i]) \\ &= \text{lcp}(\text{Suffix}(k + 1), \text{Suffix}(i)) \\ &= h[i - 1] - 1. \end{aligned}$$

Vì $h[i] = \text{height}[\text{Rank}[i]] = \text{LCP}(\text{Rank}[i] - 1, \text{Rank}[i])$, cuối cùng ta có

$$h[i] \geq h[i - 1] - 1.$$

Theo Tính chất 3, có thể cho i chạy từ 1 đến n và lần lượt tính $h[i]$ như sau.

- Nếu $\text{Rank}[i] = 1$, đặt $h[i] = 0$. Số lần so sánh ký tự là 0.
- Nếu $i = 1$ hoặc $h[i - 1] \leq 1$, trực tiếp so sánh tuần tự $\text{Suffix}(i)$ với $\text{Suffix}(\text{SA}[\text{Rank}[i] - 1])$ từ ký tự đầu tiên cho đến khi có hai ký tự khác nhau, từ đó tính $h[i]$. Số lần so sánh ký tự là $h[i] + 1$, không vượt quá $h[i] - h[i - 1] + 2$.
- Ngược lại, $i > 1$, $\text{Rank}[i] > 1$, và $h[i - 1] > 1$. Theo Tính chất 3, $\text{Suffix}(i)$ và $\text{Suffix}(\text{SA}[\text{Rank}[i] - 1])$ có ít nhất $h[i - 1] - 1$ ký tự đầu bằng nhau, nên việc so sánh ký tự có thể bắt đầu từ vị trí $h[i - 1]$ cho đến khi gặp hai ký tự khác nhau. Khi đó tính được $h[i]$, và số lần so sánh ký tự là $h[i] - h[i - 1] + 2$.

Đặt $\text{SA}[1] = p$. Không khó thấy tổng số lần so sánh ký tự không vượt quá

$$\begin{aligned} h[1] + 1 + \sum_{i=2}^{p-1} (h[i] - h[i - 1] + 2) + h[p + 1] + 1 + \sum_{i=p+2}^n (h[i] - h[i - 1] + 2) \\ \leq 1 + h[p - 1] + 2(p - 2) + h[n] + 2 + 2(n - p) \leq 4n. \end{aligned}$$

Vì vậy độ phức tạp của toàn bộ thuật toán là $\mathcal{O}(n)$. Sau khi có mảng h , theo quan hệ $\text{height}[i] = h[\text{SA}[i]]$, ta tính được mảng height trong thời gian $\mathcal{O}(n)$. Do đó

có thể tính mảng height trong thời gian $\mathcal{O}(n)$.

Kết hợp với phương pháp RMQ, sau khi tiền xử lý trong thời gian và bộ nhớ $\mathcal{O}(n)$, ta có thể tính $\text{LCP}(i, j)$ cho mọi cặp (i, j) trong thời gian hằng số.

Vì

$$\text{lcp}(\text{Suffix}(i), \text{Suffix}(j)) = \text{LCP}(\text{Rank}[i], \text{Rank}[j]),$$

ta cũng có thể tìm tiền tố chung dài nhất của hai hậu tố bất kỳ của S trong thời gian hằng số. Đây chính là một trong những nguyên nhân quan trọng khiến mảng hậu tố xử lý rất mạnh nhiều bài toán xâu.

5 Ứng dụng của mảng hậu tố

Dưới đây ta bàn cách dùng mảng hậu tố qua hai ví dụ.

5.1 Ví dụ 1: so khớp nhiều mẫu

Cho một xâu cần so khớp cố định S , độ dài n . Mỗi lần nhập một xâu mẫu P , độ dài m , yêu cầu trả về một vị trí khớp của P trong S , hoặc báo khớp thất bại. Một lần khớp nghĩa là có vị trí i thỏa

$$1 \leq i \leq n - m + 1 \quad \text{và} \quad S[i..(i + m - 1)] = P,$$

tức $\text{Suffix}(i) =_m P$.

Ta biết nếu chỉ có một xâu mẫu, thuật toán tốt nhất là KMP, với thời gian $\mathcal{O}(n + m)$. Nhưng nếu có nhiều xâu mẫu, ta nên tiền xử lý thích hợp để giảm thời gian so khớp cho mỗi mẫu.

Cách tiền xử lý đơn giản nhất là xây dựng mảng hậu tố của S sau khi thêm $\$$ vào cuối S . Khi đó mỗi lần tìm khớp được chuyển thành bài toán dùng tìm kiếm nhị phân trên SA để tìm một hậu tố có tiền tố chung dài nhất với P , rồi kiểm tra độ dài tiền tố chung dài nhất ấy có bằng m hay không.

Theo cách này, mỗi lần so sánh P với một hậu tố mất $\mathcal{O}(m)$, vì trong trường hợp xấu nhất có thể phải so sánh m ký tự. Tìm kiếm nhị phân gọi phép so sánh $\mathcal{O}(\log n)$ lần, nên tổng độ phức tạp là $\mathcal{O}(m \log n)$. Như vậy độ phức tạp mỗi lần so khớp giảm từ $\mathcal{O}(n + m)$ xuống $\mathcal{O}(m \log n)$, đã cải thiện đáng kể.

Nhưng như thế vẫn chưa đủ thỏa mãn. Ở trên đã nói LCP có thể tăng sức mạnh của mảng hậu tố; ta thử dùng nó trong bài toán này.

Ta phân tích thuật toán tìm kiếm nhị phân ban đầu. Đại thể gồm các bước sau.

- Step 1.** Đặt $left = 1, right = n, max_match = 0$.
- Step 2.** Đặt $mid = (left + right)/2$, trong đó “/” là chia lấy phần nguyên.
- Step 3.** So sánh tuần tự các ký tự tương ứng của Suffix(SA[mid]) và P , tìm tiền tố chung dài nhất r của chúng, đồng thời xác định quan hệ lớn nhỏ. Nếu $r > max_match$, đặt $max_match = r$ và $ans = mid$.
- Step 4.** Nếu Suffix(SA[mid]) $< P$, đặt $left = mid + 1$; nếu Suffix(SA[mid]) $> P$, đặt $right = mid - 1$; nếu Suffix(SA[mid]) $= P$, chuyển đến Step 6.
- Step 5.** Nếu $left < right$, quay lại Step 2; nếu không thì đến Step 6.
- Step 6.** Nếu $max_match = m$, xuất ans ; nếu không, xuất “không khớp”.

Sự chú ý nhanh chóng dồn vào Step 3. Nếu tránh được việc lần nào cũng so sánh Suffix(SA[mid]) và P từ đầu, có lẽ độ phức tạp còn giảm được nữa.

Tương tự quá trình tính mảng height, ta xét việc dùng tiền tố chung dài nhất đã biết trước đó làm “nền” cho so sánh, nhằm tránh các phép so sánh ký tự dư thừa.

Trước khi so sánh Suffix(SA[mid]) với P , ta tính LCP(mid, ans) trong thời gian hằng số, rồi so sánh LCP(mid, ans) với max_match .

Trường hợp 1. Nếu LCP(mid, ans) $< max_match$, thì tiền tố chung dài nhất của Suffix(SA[mid]) và P chính là LCP(mid, ans). Nói cách khác, ta trực tiếp xác định được $r = \text{LCP}(mid, ans)$ trong Step 3, rồi chỉ cần so sánh ký tự thứ $r + 1$ của hai xâu. Kết quả chắc chắn không bằng nhau và đủ để xác định quan hệ lớn nhỏ giữa Suffix(SA[mid]) và P . Trong trường hợp này số lần so sánh ký tự là 1.

Trường hợp 2. Nếu LCP(mid, ans) $\geq max_match$, thì Suffix(SA[mid]) và Suffix(SA[ans]) chắc chắn có max_match ký tự đầu bằng nhau. Do đó Suffix(SA[mid]) và P cũng có max_match ký tự đầu bằng nhau. Ta có thể bắt đầu so sánh từ ký tự thứ $max_match + 1$. Giá trị r thu được cuối cùng chắc chắn không nhỏ hơn max_match , và số lần so sánh ký tự là

$$r - max_match + 1.$$

Để thấy sau khi Step 3 thực hiện xong, max_match sẽ bằng r .

Gọi lượng tăng của max_match sau mỗi lần thực hiện Step 3 là $\Delta_{\max}$. Trong Trường hợp 1, $\Delta_{\max} = 0$ và số lần so sánh ký tự là $1 = \Delta_{\max} + 1$. Trong Trường hợp 2, $\Delta_{\max} = r - max_match$, và số lần so sánh ký tự cũng là $\Delta_{\max} + 1$. Tóm lại, mỗi lần thực hiện Step 3, số lần so sánh ký tự là $\Delta_{\max} + 1$.

Tổng số lần so sánh ký tự bằng tổng mọi $\Delta_{\max}$ cộng với số lần thực hiện Step 3. Tổng mọi $\Delta_{\max}$ rõ ràng chính là giá trị max_match cuối cùng, không vượt quá len(P) $= m$, còn số lần thực hiện Step 3 là

$\mathcal{O}(\log n)$. Vì vậy tổng số lần so sánh ký tự là $\mathcal{O}(m + \log n)$. Toàn bộ thuật toán hiển nhiên cùng bậc với số lần so sánh ký tự, nên độ phức tạp là $\mathcal{O}(m + \log n)$.

Đến đây bài toán được giải quyết trọn vẹn. Sau thời gian tiền xử lý $\mathcal{O}(n \log n)$, gồm xây dựng mảng hậu tố, mảng hạng, tính mảng height và tiền xử lý RMQ, ta có thể so khớp một xâu mẫu P độ dài m trong thời gian $\mathcal{O}(m + \log n)$. Đây chỉ là chi phí đọc P cộng thêm $\log n$, một kết quả rất tốt.

5.2 Ví dụ 2: xâu con đối xứng dài nhất

Một xâu đối xứng là một xâu u thỏa

$$u[i] = u[\text{len}(u) - i + 1], \quad 1 \leq i \leq \text{len}(u).$$

Nói cách khác, xâu đối xứng u “đối xứng” quanh vị trí giữa của nó.

Theo tính chẵn lẻ của độ dài, xâu đối xứng được chia làm hai loại: xâu đối xứng độ dài lẻ gọi là xâu đối xứng lẻ, xâu đối xứng độ dài chẵn gọi là xâu đối xứng chẵn.

Hãy tưởng tượng ta kẻ một đường thẳng qua “vị trí trung tâm” của xâu đối xứng u . Với xâu đối xứng lẻ, đường này đi qua ký tự chính giữa $u[(\text{len}(u) + 1)/2]$; với xâu đối xứng chẵn, đường này đi giữa hai ký tự giữa $u[\text{len}(u)/2]$ và $u[\text{len}(u)/2 + 1]$. Hai ví dụ trong bản gốc có thể biểu diễn như sau.

$$a \ b \ \boxed{c} \ b \ a \qquad c \ a \ l \ f \mid f \ l \ a \ c$$

Các ký tự trong xâu đối xứng phân bố đối xứng qua đường trung tâm này. Xâu bên trái đường trung tâm, khi đảo ngược, bằng xâu bên phải; ta gọi hai phần đó là “bằng nhau qua phản chiếu”.

Xâu con đối xứng của xâu T là những xâu con của T đồng thời là xâu đối xứng. Trong các xâu con đối xứng của T , xâu có độ dài lớn nhất gọi là xâu con đối xứng dài nhất. Tương tự, ta định nghĩa xâu con đối xứng lẻ/chẵn và xâu con đối xứng lẻ/chẵn dài nhất.

Bài toán xâu con đối xứng dài nhất: cho một xâu T , tìm xâu con đối xứng dài nhất của T . Để đơn giản, ta chỉ yêu cầu đưa ra độ dài của xâu con đối xứng dài nhất.

Dưới đây ta phân tích cách tìm xâu con đối xứng lẻ dài nhất; cách tìm xâu con đối xứng chẵn dài nhất tương tự.

Mỗi xâu con đối xứng lẻ chắc chắn có một vị trí trung tâm, tức ký tự chính giữa của toàn bộ xâu đối xứng. Vị trí trung tâm này là một ký tự nào đó của T , vì vậy trước hết ta liệt kê vị trí trung tâm. Với một vị trí trung tâm cố định i , có thể có nhiều xâu con đối xứng lẻ lấy $T[i]$ làm tâm, nhưng chúng có một tính chất: phần bên trái và bên phải của $T[i]$ có độ dài bằng nhau và đối xứng qua $T[i]$, tức hai ký tự trái-phải cách $T[i]$ cùng một khoảng thì bằng nhau.

Vì vậy mọi xâu con đối xứng lẻ có tâm $T[i]$ đều có dạng

$$T[i - r..i + r], \quad r \geq 0.$$

Ta thấy r càng lớn thì xâu đối xứng lấy $T[i]$ làm tâm càng dài. Do đó chỉ cần tìm r lớn nhất sao cho $T[i - r..i - 1]$ và $T[i + 1..i + r]$ đối xứng qua $T[i]$, ta sẽ tìm được xâu con đối xứng lẻ dài nhất có tâm $T[i]$; hơn nữa xâu này rõ ràng là duy nhất. Nếu liệt kê i từ 1 đến $\text{len}(T)$, tìm xâu con đối xứng lẻ dài nhất có tâm từng $T[i]$, rồi lấy xâu dài nhất trong số đó, ta thu được xâu con đối xứng lẻ dài nhất của toàn bộ T .

Làm thế nào tìm xâu con đối xứng lẻ dài nhất có tâm $T[i]$? Trước hết, khi $r = 0$, riêng ký tự $T[i]$ đã tạo thành một xâu con đối xứng lẻ, độ dài 1. Tiếp theo ta xét tăng dần r . Giả sử khi $r = k$, $T[i - r..i + r]$ là xâu con đối xứng lẻ, tức $T[i - r..i - 1]$ và $T[i + 1..i + r]$ bằng nhau qua phản chiếu. Khi $r = k + 1$, ta so sánh hai ký tự $T[i - (k + 1)]$ và $T[i + k + 1]$. Nếu chúng bằng nhau, thì $T[i - (k + 1)..i - 1]$ và $T[i + 1..i + k + 1]$ đối xứng qua $T[i]$, vì theo giả thiết $T[i - k..i - 1]$ và $T[i + 1..i + k]$ đã đối xứng qua $T[i]$.

Khi đó r có thể mở rộng tới $k + 1$. Nếu $T[i - (k + 1)]$ và $T[i + k + 1]$ khác nhau, thì $T[i - (k + 1)..i - 1]$ và $T[i + 1..i + k + 1]$ không đối xứng; đồng thời với mọi $r' > k + 1$, $T[i - r'..i - 1]$ và $T[i + 1..i + r']$ cũng không thể đối xứng qua $T[i]$. Vì vậy r lớn nhất chỉ có thể là k .

Ta gọi quá trình tăng r là mở rộng sang hai phía. Mỗi lần mở rộng làm độ dài của xâu con đối xứng lẻ tâm $T[i]$ tăng thêm 2. Giá trị lớn nhất mà r mở rộng tới quyết định độ dài của xâu con đối xứng lẻ dài nhất có tâm $T[i]$, bằng $2r + 1$.

Đặt $\text{len}(T) = m$. Nếu dùng cách so sánh tuần tự các ký tự tương ứng để tìm giá trị mở rộng lớn nhất sang hai phía, ta có thể phải so sánh nhiều nhất $m - 1$ ký tự. Vì cần liệt kê từng vị trí làm tâm và mở rộng sang hai phía, độ phức tạp trường hợp xấu nhất có thể đạt $\mathcal{O}(m^2)$, chưa lý tưởng. Dưới đây ta tối ưu phần lõi của thuật toán: tìm giá trị mở rộng lớn nhất quanh một vị trí tâm.

Thêm một ký tự đặc biệt $\#$ vào cuối T , quy định nó khác mọi ký tự của T . Sau đó đảo ngược T , nối sau $\#$, rồi thêm một ký tự đặc biệt $\$$ vào cuối xâu T' , quy định nó nhỏ hơn mọi ký tự đứng trước. Xâu ghép thu được gọi là S .

Sơ đồ trong bản gốc là ví dụ với $T = \text{banana}$:

$$\begin{array}{c} b \ a \ n \ a \ n \ a \ | \ \# \ | \ a \ n \ a \ n \ a \ b \ | \ \$ \\ T \qquad \qquad \qquad T' \end{array}$$

Nếu i là vị trí trong phần T , thì vị trí đối xứng của nó trong phần T' là

$$i' = 2m - i + 2.$$

Dễ thấy mọi ký tự trong T đều tìm được một ký tự bằng nó ở vị trí đối xứng trong T' . Nếu dùng chỉ số của S , thì $S[1..m]$ là T , còn $S[m + 2..2m + 1]$ là T' . Khi đó với mỗi $S[i]$, $1 \leq i \leq m$, ký tự đối xứng của nó qua $\#$ là $S[2m - i + 2]$. Tương tự, một xâu con

$$S[i..j] \quad (1 \leq i \leq j \leq m)$$

trong T cũng tìm được một xâu con bằng nó qua phản chiếu:

$$S[2m - j + 2..2m - i + 2].$$

Bây giờ cố định một vị trí $S[i]$ trong T làm tâm. Giả sử đã mở rộng sang hai phía tới $i - r$ và $i + r$. Khi đó $S[i - r..i - 1]$ và $S[i + 1..i + r]$ bằng nhau qua phản chiếu. Ký tự $S[i]$ có ký tự đối xứng $S[2m - i + 2]$ trong T' . Đặt $i' = 2m - i + 2$. Khi đó $S[i - r..i - 1]$ cũng tìm được xâu con đối xứng trong T' là

$$S[i' + 1..i' + r].$$

Do $S[i + 1..i + r]$ và $S[i' + 1..i' + r]$ cùng bằng $S[i - r..i - 1]$ qua phản chiếu, ta có

$$S[i + 1..i + r] = S[i' + 1..i' + r].$$

Lại vì $S[i] = S[i']$, nên

$$S[i..i + r] = S[i'..i' + r],$$

tức

$$\text{Suffix}(i) =_{r+1} \text{Suffix}(i').$$

Nay ta muốn r càng lớn càng tốt, tức cần tính

$$\max\{r \mid \text{Suffix}(i) =_{r+1} \text{Suffix}(i')\}.$$

Để thấy ở đây

$$r = \text{LCP}(\text{Rank}[i], \text{Rank}[i']) - 1.$$

Lập luận trên còn một điểm cần lưu ý: giá trị $\text{LCP}(\text{Rank}[i], \text{Rank}[i']) - 1$ mới chỉ có thể xem là một cận trên của r , chưa thể coi ngay là giá trị lớn nhất của r , vì còn phải chứng minh rằng tiền tố chung dài nhất của $\text{Suffix}(i)$ và $\text{Suffix}(i')$ chắc chắn cho ta ngược lại một xâu đối xứng tương ứng có tâm i trong T . Chứng minh này tương tự lập luận ở trên; điểm cần chú ý là ký tự đặc biệt $\#$ được dùng để tránh nguy cơ $\text{LCP}(\text{Rank}[i], \text{Rank}[i'])$ vượt quá giá trị r thực sự lớn nhất. Phần chứng minh ấy để lại cho bạn đọc.

Tóm lại, ta đã xác định rằng tìm giá trị mở rộng lớn nhất sang hai phía quanh tâm $T[i]$ tương đương với tính LCP của hai hậu tố bắt đầu tại i và i' . Theo các nội dung trước về mảng hậu tố và LCP, thao tác này thực hiện được trong thời gian hằng số, miễn là ta đã bỏ ra $\mathcal{O}(n \log n)$ để tính mảng hậu tố, mảng height và tiền xử lý, trong đó

$$n = \text{len}(S) = 2m + 2.$$

Bây giờ độ dài của xâu con đối xứng dài nhất có tâm tại mỗi vị trí $T[i]$ có thể tính trong thời gian hằng số. Ta liệt kê i từ 1 đến m , lần lượt tính các giá trị này và ghi nhận độ dài lớn nhất; đó chính là độ dài xâu con đối xứng lẻ dài nhất cần tìm. Vì mỗi tâm chỉ mất thời gian hằng số, tổng độ phức tạp của phần liệt kê là $\mathcal{O}(m)$.

Do đó độ phức tạp toàn bộ thuật toán là

$$\mathcal{O}(n \log n + m) = \mathcal{O}(2m \log(2m) + m) = \mathcal{O}(m \log m),$$

một thuật toán rất tốt, cải thiện lớn so với thuật toán bậc hai trước đó.

6 So sánh mảng hậu tố và cây hậu tố

Qua hai ví dụ trên, hẳn người đọc đã thấy phần nào sức mạnh của mảng hậu tố. Một cấu trúc dữ liệu khác, cây hậu tố, cũng có thể dùng cho các bài toán này. Vậy mảng hậu tố và cây hậu tố khác nhau, liên hệ với nhau ra sao? Ta so sánh như sau.

Trước hết, mảng hậu tố dễ hiểu hơn, cũng dễ lập trình hơn. Nó không cần thao tác con trỏ như cây hậu tố, nên việc gỡ lỗi thuận tiện hơn.

Thứ hai, mảng hậu tố dùng ít bộ nhớ hơn cây hậu tố. Trong phần phân tích trên ta chưa nhắc đến độ phức tạp bộ nhớ, giờ nói ngắn gọn như sau. Mảng hậu tố SA và mảng hạng Rank mỗi mảng chỉ cần n số nguyên. Trong quá trình tính SA_{2k} từ Rank_k , cần hai mảng một chiều phụ trợ, mỗi mảng chiếm n số nguyên, và dùng chúng theo kiểu cuốn chiếu. Toàn bộ thuật toán chỉ cần bốn mảng một chiều này cùng hằng số biến phụ, nên tổng bộ nhớ là $4n$ số nguyên.

Trong khi đó, cây hậu tố thường có hơn $2n$ đỉnh. Thông thường mỗi đỉnh cần hai số nguyên; ngay cả khi dùng một số kỹ thuật, ít nhất vẫn phải lưu một số nguyên. Mỗi đỉnh còn cần hai con trỏ nếu dùng cách biểu diễn con-trái anh-em. Vì thế tổng bộ nhớ tối thiểu là $4n$ con trỏ và $2n$ số nguyên; ít nhất cũng phải n số nguyên. Nếu dùng cách khác để biểu diễn cấu trúc cây, bộ nhớ cần thiết còn lớn hơn.

Có thể thấy nhu cầu bộ nhớ của mảng hậu tố nhỏ hơn cây hậu tố.

Cuối cùng, ta so sánh độ phức tạp. Trước hết chia bảng chữ cái Σ theo tổng số ký tự $|\Sigma|$ thành ba loại:

- nếu $|\Sigma|$ là hằng số, gọi Σ là bảng chữ cái hằng số (*Constant Alphabet*);
- nếu $|\Sigma|$ là một hàm đa thức theo độ dài n của S , gọi Σ là bảng chữ cái nguyên (*Integer Alphabet*);
- nếu $|\Sigma|$ không bị giới hạn về kích thước, gọi Σ là bảng chữ cái tổng quát (*General Alphabet*).

Dễ thấy bảng chữ cái hằng số là một trường hợp của bảng chữ cái nguyên, còn bảng chữ cái nguyên là một trường hợp của bảng chữ cái tổng quát.

Độ phức tạp xây dựng mảng hậu tố không phụ thuộc vào bảng chữ cái, vì thuật toán trực tiếp xử lý bảng chữ cái tổng quát.

Với phương pháp thông thường để xây dựng cây hậu tố, nếu dùng biểu diễn cây bằng con-trái anh-em, độ phức tạp thời gian đạt $\mathcal{O}(n|\Sigma|)$. Rõ ràng cách này rất kém hiệu quả với bảng chữ cái nguyên và bảng chữ cái tổng quát, và cũng không phù hợp với bảng chữ cái hằng số có $|\Sigma|$ khá lớn.

Cách khắc phục là dùng cây nhị phân cân bằng để lưu các con trở tới con, khi đó độ phức tạp trở thành $\mathcal{O}(n \log |\Sigma|)$.

Như vậy trong một số trường hợp, cây hậu tố có ưu thế tốc độ so với mảng hậu tố, nhưng ưu thế không quá rõ. Với các xâu có $|\Sigma|$ rất nhỏ, ưu thế tốc độ của cây hậu tố so với mảng hậu tố là đáng kể, đặc biệt với xâu nhị phân 0-1, vốn rất thường gặp.

Mảng hậu tố thực chất có thể xem là mảng thu được bằng cách xếp toàn bộ lá của cây hậu tố theo thứ tự từ trái sang phải. Vì vậy phạm vi ứng dụng của mảng hậu tố không thể vượt quá cây hậu tố. Thậm chí có thể nói rằng nếu không kết hợp với LCP, phạm vi ứng dụng của mảng hậu tố khá hẹp. Nhưng khi kết hợp với hàm LCP, mảng hậu tố trở nên rất mạnh và có thể hoàn thành phần lớn công việc mà cây hậu tố làm được, bởi hàm LCP thực chất cho ta tổ tiên chung gần nhất của hai lá bất kỳ. Phần này bạn đọc có thể tự nghiên cứu thêm.

Cây hậu tố và mảng hậu tố đều là những cấu trúc dữ liệu rất xuất sắc trong xử lý xâu. Không thể nói cấu trúc nào chắc chắn tốt hơn cấu trúc nào; với các bài toán ở hoàn cảnh và điều kiện khác nhau, ta nên vận dụng linh hoạt, lựa chọn cẩn thận cấu trúc phù hợp hơn. Thuật toán và cấu trúc dữ liệu đều là vật tĩnh; người vận dụng chúng mới là nhân vật chính. Nắm vững các thuật toán và cấu trúc dữ liệu kinh điển, rồi dùng chúng đúng chỗ để phát huy sức mạnh lớn nhất, đó mới là trí tuệ quan trọng trong nghiên cứu và thi đấu tin học, cũng là sức hấp dẫn của các kỳ thi tin học.